



FPT POLYTECHNIC



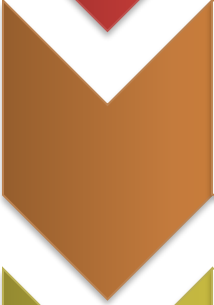
Conceive Design Implement Operate

BÀI 7: KỸ THUẬT TƯƠNG TÁC DỮ LIỆU

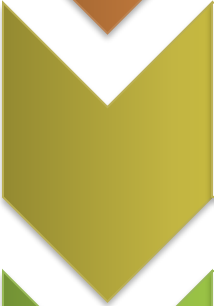
www.poly.edu.vn



Coordinated Multiple Views



Linking & Brushing



Python Dash



User Experience



Tại sao cần tương tác?

- Khám phá sâu: Dữ liệu đa chiều không thể biểu diễn hết trên một biểu đồ đơn lẻ
- Giảm nhiễu: Tương tác giúp người dùng lọc bỏ thông tin thừa, tập trung vào vùng nghi vấn
- Xác thực: Giúp con người hiểu và kiểm chứng kết quả của các mô hình "hộp đen"



1. MULTIPLE VIEWS

Coordinated Multiple Views (CMV)

CMV là kỹ thuật sử dụng hai hoặc nhiều view đồng thời để hỗ trợ việc tìm kiếm mẫu dữ liệu phức tạp.

- **Sự đa dạng:** Mỗi biểu đồ cung cấp một góc nhìn khác nhau (Time-series, Distribution, Map)
- **Sự phối hợp:** Hành động trên một view ảnh hưởng đến các view còn lại

Không gian thiết kế Multiple Views

Khía cạnh	Lựa chọn thiết kế	Mục đích
Bố cục	Tiled, Overlaid, Small Multiples	Tối ưu hóa không gian màn hình
Nội dung	Same data, Different data, Subset	So sánh đa chiều
Tương quan	Overview + Detail, Master + Slave	Điều hướng dữ liệu lớn

Phương pháp: Overview + Detail

Một view chứa toàn bộ dữ liệu (Context), một view khác chứa vùng phóng to (Focus)

- > Giảm tải bộ nhớ thị giác (Visual working memory)
- > Duy trì định hướng trong không gian dữ liệu khổng lồ

Phương pháp: Small Multiples (Trellis)

Sử dụng cùng một loại biểu đồ cho các tập con khác nhau của dữ liệu

- > Giúp so sánh cực kỳ hiệu quả qua các danh mục
- > Dễ dàng nhận diện các điểm dị biệt (outliers) giữa các nhóm

Nguyên tắc bố cục Dashboard

- **Quy tắc 5 giây:** Người dùng phải hiểu mục đích dashboard trong vòng 5 giây đầu tiên
- **Phân cấp thị giác:** Thông tin quan trọng nhất nằm ở góc trên bên trái (Z-pattern)

Nguyên tắc bố cục Dashboard

- **Quy tắc 5 giây:** Người dùng phải hiểu mục đích dashboard trong vòng 5 giây đầu tiên
- **Phân cấp thị giác:** Thông tin quan trọng nhất nằm ở góc trên bên trái (Z-pattern)



2. KỸ THUẬT LIÊN KẾT

Kỹ thuật Brushing

Brushing là hành động chọn một tập con các thực thể dữ liệu bằng thiết bị đầu vào (chuột, chạm)

Các loại Brushing phổ biến:

- **Point selection:** Chọn từng điểm đơn lẻ
- **Box/Rect brush:** Kéo vùng hình chữ nhật
- **Lasso brush:** Khoanh vùng tự do

Kỹ thuật Linking

Linking là việc truyền bá (propagate) trạng thái lựa chọn từ view này sang view khác

Các loại Linking phổ biến

- **Visual Linking:** Thay đổi thuộc tính hiển thị (màu sắc, độ mờ) của các điểm tương ứng trên view khác
- **Data Linking:** Lọc dữ liệu (Filter) trên các biểu đồ khác dựa trên tiêu chí đã chọn

Các kịch bản tương tác

Kỹ thuật	Hành động người dùng	Phản hồi hệ thống
Brush-to-Filter	Chọn vùng trên Scatter plot	Histogram chỉ hiện phân phối vùng đó
Brush-to-Label	Di chuột qua điểm dữ liệu	Hiển thị Tooltip chi tiết
Cross-filtering	Thay đổi Slicer (thanh trượt)	Toàn bộ dashboard cập nhật theo tập con

Thách thức trong Linking/Brushing

- Độ trễ (Latency): Khi dữ liệu lớn, việc tính toán filter cho mọi biểu đồ gây giật lag
- Nhiều thị giác: Quá nhiều màu sắc khi highlight gây rối mắt
- Mất ngữ cảnh: Khi filter quá sâu, người dùng quên mất tổng thể ban đầu

Tóm tắt Multiple View & Linking

Khái niệm	Từ khóa chính	Lưu ý quan trọng
Multiple Views	Coordinated, Context, Complementary	Tránh gây quá tải thị giác.
Brushing	Selection, Interaction, Subset	Hỗ trợ nhiều kiểu chọn (box, lasso).
Linking	Propagation, Synchronization	Phản hồi phải nhanh ($< 100\text{ms}$).



3. PYTHON DASH

Giới thiệu

Dash là framework mã nguồn mở của Plotly để xây dựng ứng dụng phân tích dữ liệu trên nền web bằng Python

- Không cần JavaScript/HTML/CSS chuyên sâu
- Tích hợp hoàn hảo với Pandas, Scikit-learn, PyTorch
- Hỗ trợ tính tương tác cao thông qua cơ chế Callback

Kiến trúc của ứng dụng Dash

1. Layout

Xác định cấu trúc giao diện (Biểu đồ, Dropdown, Slider) bằng dash.html và dash.dcc

2. Callbacks

Logic tương tác: Hàm Python sẽ chạy khi dữ liệu đầu vào (Input) thay đổi và cập nhật giao diện (Output)

-> *Flask (Backend) + React.js (Frontend) + Plotly.js (Graphics)*

Các thành phần cốt lõi (dcc)

Component	Chức năng	Thuộc tính tương tác quan trọng
dcc.Graph	Hiển thị biểu đồ Plotly	hoverData, clickData, selectedData
dcc.Dropdown	Chọn danh mục	value
dcc.Slider	Chọn dải giá trị	value

Cơ chế Callback trong Dash

- Callback là cơ chế cho phép ứng dụng tự động cập nhật giao diện khi dữ liệu đầu vào thay đổi
- Callback = Hàm Python được Dash gọi khi Input thay đổi để cập nhật Output
- User Interaction - > Input thay đổi - > Dash gọi Callback Function - > Function xử lý dữ liệu - > Output Component cập nhật

Triển khai Brushing trong Dash

Khi người dùng khoanh vùng (box select) trên biểu đồ A

- Dash bắt sự kiện `selectedData`
- Trích xuất tọa độ hoặc Index của điểm dữ liệu
- Lọc tập dữ liệu gốc bằng Pandas
- Cập nhật figure của biểu đồ B

Callback với nhiều đầu vào

Cho phép một biểu đồ thay đổi dựa trên nhiều tương tác cùng lúc

- Input 1: Vùng chọn trên bản đồ (Geography)
- Input 2: Thanh trượt thời gian (Time)
- Input 3: Dropdown loại sản phẩm (Category)

Tối ưu hóa hiệu năng (Big Data)

- Client-side Callbacks: Chạy logic tương tác bằng JavaScript trên trình duyệt thay vì gửi về server Python
- Data Caching: Sử dụng Redis hoặc Flask-Caching để lưu trữ kết quả tính toán trung gian
- Patching: Chỉ gửi phần thay đổi của biểu đồ (như màu sắc) thay vì gửi lại toàn bộ dữ liệu

Phân biệt Input và State

Loại	Kích hoạt Callback?	Mục đích
Input	Có (Ngay lập tức)	Tương tác thời gian thực (Slider, Click).
State	Không	Đọc giá trị hiện tại (ví dụ: Textbox) khi bấm nút Submit.

- > Dùng State giúp giảm số lượng tính toán không cần thiết

Ví dụ: Cross-filtering Dashboard

Xây dựng dashboard phân tích doanh số:

- Scatter plot: Giá vs Lợi nhuận
- Bar chart: Doanh số theo danh mục

Yêu cầu: Khi khoanh vùng trên Scatter plot, Bar chart phải tự động lọc theo danh mục của các điểm được chọn

Ví dụ: Cross-filtering Dashboard

1. Chuẩn bị thư viện và dữ liệu



```
import pandas as pd
```

```
data = {  
    "price": [10, 20, 30, 15, 25, 35, 40],  
    "profit": [2, 5, 8, 3, 6, 9, 10],  
    "category": ["A", "A", "B", "B", "C", "C", "A"],  
    "sales": [100, 120, 90, 80, 150, 130, 160]  
}
```

```
df = pd.DataFrame(data)
```



```
import dash  
from dash import dcc, html  
from dash.dependencies import Input, Output  
  
import plotly.express as px
```

Ví dụ: Cross-filtering Dashboard

2. Xây dựng Layout

- layout mô tả cấu trúc HTML của trang web
- Div (container)
 - |
 - ├── H2 (title)
 - ├── Graph (scatter plot)
 - └── Graph (bar chart)



```
app = dash.Dash(__name__)

app.layout = html.Div([

    html.H2("Sales Cross Filtering Dashboard"),

    dcc.Graph(id="scatter-chart"),

    dcc.Graph(id="bar-chart")

])
```

Ví dụ: Cross-filtering Dashboard

3. Tạo Scatter-Plot

- Mỗi điểm = 1 sản phẩm
- Input → scatter-chart.id
- Output → scatter-chart.figure

Trục	Dữ liệu
X	price
Y	profit
Color	category

```
@app.callback(  
    Output("scatter-chart", "figure"),  
    Input("scatter-chart", "id")  
)  
def display_scatter(_):  
  
    fig = px.scatter(  
        df,  
        x="price",  
        y="profit",  
        color="category"  
    )  
  
    return fig
```

Ví dụ: Cross-filtering Dashboard

4. Cross-Filtering Callback

- Input: selectedData - > Chứa các điểm được chọn bằng box/lasso select
- Nếu chưa chọn điểm - > Bar chart hiển thị toàn bộ dữ liệu
- Nếu chọn điểm - > Lấy index của các điểm được chọn → Lọc dataframe

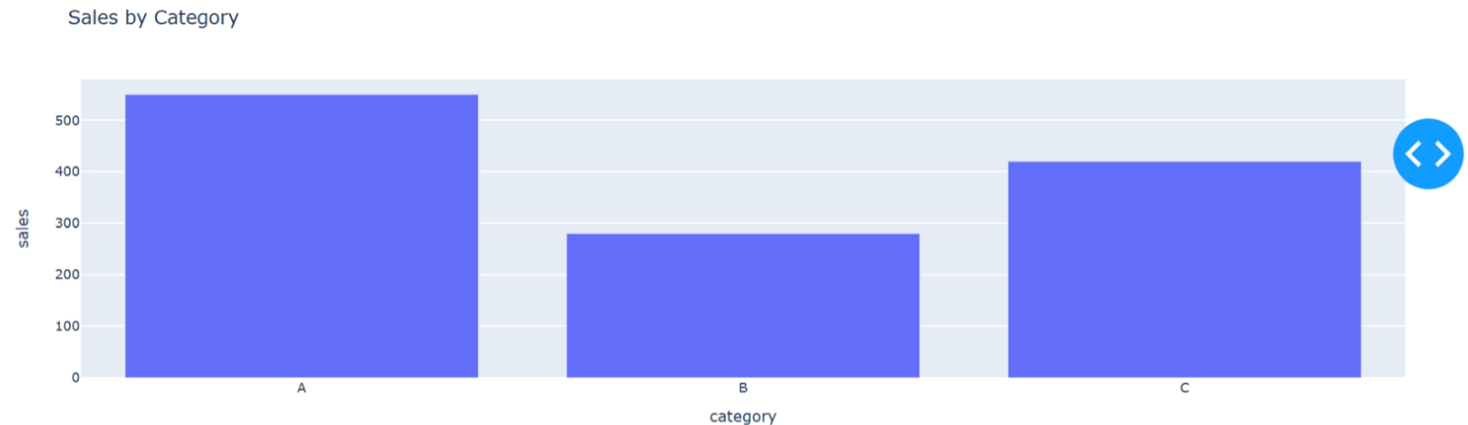
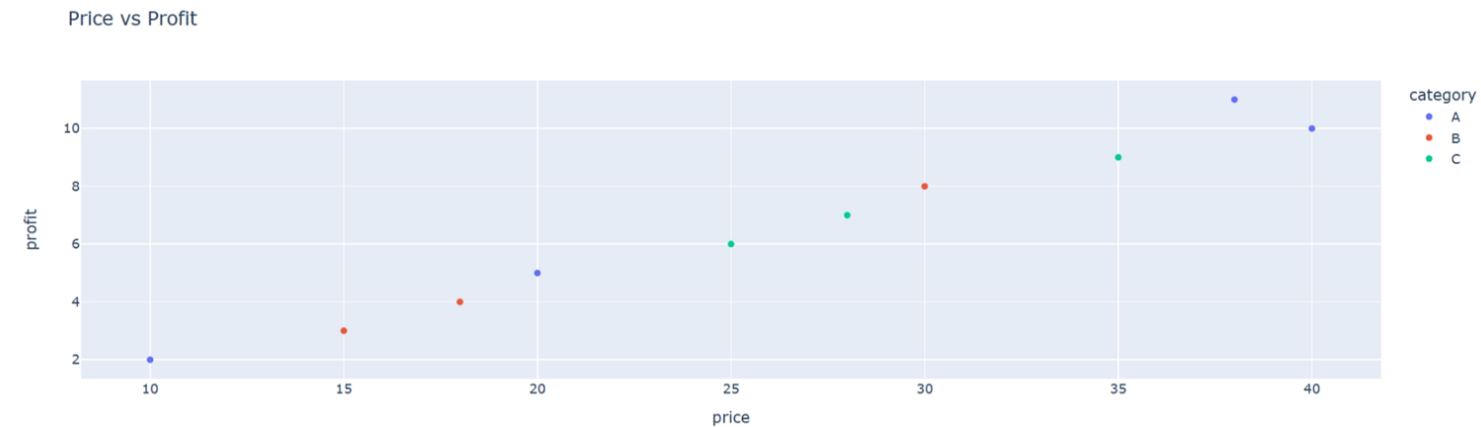
```
@app.callback(  
    Output("bar-chart", "figure"),  
    Input("scatter-chart", "selectedData")  
)  
def update_bar(selectedData):  
    if selectedData is None:  
        filtered = df  
    else:  
        points = [p["pointIndex"] for p in  
selectedData["points"]]  
        filtered = df.iloc[points]  
  
        sales_by_cat = filtered.groupby("category")  
["sales"].sum().reset_index()  
  
        fig = px.bar(  
            sales_by_cat,  
            x="category",  
            y="sales",  
            title="Sales by Category"  
        )  
  
    return fig
```

Ví dụ: Cross-filtering Dashboard

5. Khởi động ứng dụng

```
if __name__ == "__main__":  
    app.run_server(debug=True)
```

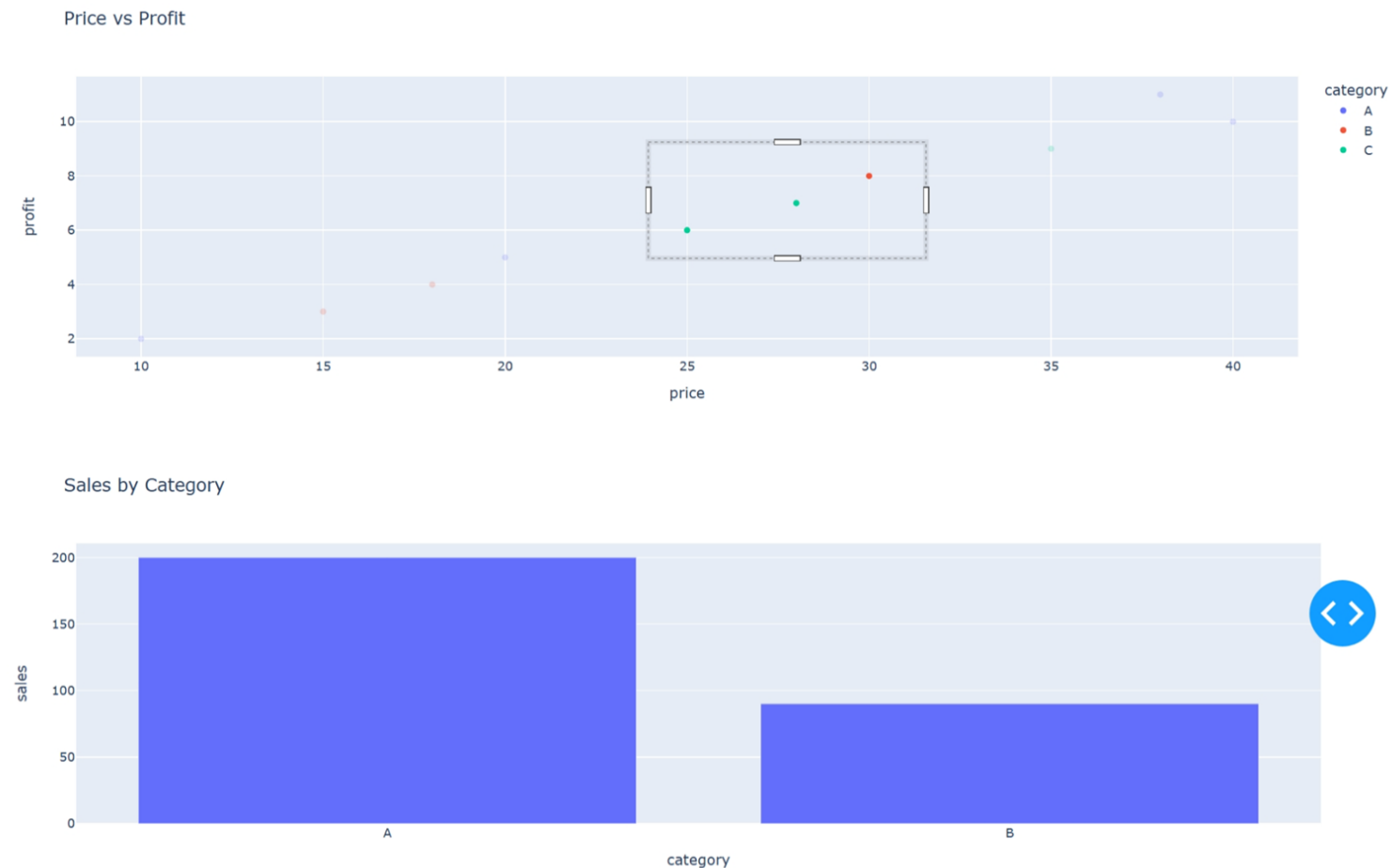
Cross Filtering Sales Dashboard



Ví dụ: Cross-filtering Dashboard

6. Khoanh vùng dữ liệu để
thấy sự thay đổi

Cross Filtering Sales Dashboard



Interactive EDA trong AI Pipeline

Tương tác đóng vai trò quan trọng trong việc gỡ lỗi mô hình AI

- Chọn các điểm có Loss cao để xem thuộc tính đặc trưng
- Brushing trên biểu đồ UMAP/t-SNE để gán nhãn dữ liệu thủ công
- Tương tác với Hyperparameters và xem kết quả Accuracy thay đổi tức thì

Thiết kế UX cho Tương tác

- **Visual Feedback:** Phần tử được chọn phải nổi bật rõ rệt. Các phần tử không chọn nên được làm mờ (ghosting)
- **Consistency:** Màu sắc giữa các biểu đồ phải đồng nhất. Ví dụ: "Apple" luôn là màu đỏ ở mọi view
- **Reversibility:** Luôn cung cấp nút "Reset" hoặc cho phép click ra vùng trắng để bỏ chọn

Dash Enterprise & Deployment

Một số lưu ý khi triển khai thực tế

- Unicorn/Nginx cho hiệu năng phục vụ web
- Docker hóa ứng dụng để triển khai trên Cloud (AWS/Azure)
- Quản lý quyền truy cập (Authentication/Authorization)



FPT POLYTECHNIC

THANK YOU

